

1) Konfigurasi & Metadata Project

``user/package.json``

- Mendefinisikan dependensi project (Expo, React, expo-router, dan library UI lain).
- Menentukan script untuk menjalankan project.

``user/app.json``

- Konfigurasi Expo (nama aplikasi, scheme, ikon, orientation, plugins).
- Contoh bagian penting: plugin ``expo-router``.

``user/eslint.config.js``

- Konfigurasi rule ESLint untuk TypeScript/React.

``user/tsconfig*.json``

- ``tsconfig.json`` / ``tsconfig.app.json`` / ``tsconfig.types.json``
- Mengatur target TypeScript, path, strictness, dan inclusion/exclusion.

``user/.gitignore``

- Mengatur file/folder yang tidak perlu masuk git (mis. build artifacts, caches).

2) Assets

``user/assets/``

- Gambar-gambar yang dipakai aplikasi:
 - ``images/tabIcons/*`` untuk icon tab bar
 - ``images/*`` untuk gambar kartu wisata dan ikon aplikasi

3) Konstanta Theme

``user/constants/theme.ts``

- Definisi tema (mis. warna, font, dsb). Dipakai bila ada komponen yang butuh referensi styling terpusat.

4) Client API (paling penting untuk integrasi backend)

``user/lib/api.ts``

- Menyediakan fungsi untuk memanggil backend NestJS.

Tipe data

- `Wisata`
- `Trip`
- `Booking`

Base URL

- Menggunakan:
 - `process.env.EXPO\_PUBLIC\_API\_BASE\_URL`
 - fallback ke `http://localhost:3000`

Fungsi request umum

- `request<T>(path, options)`
 - melakukan `fetch`
 - melempar error jika `res.ok === false`

Endpoint yang dipakai user

- `apiWisata`:
 - `getAll()` → `GET /wisata`
- `apiTrip`:
 - `getAll()` → `GET /trip`
 - `getByWisata(wisataId)` → `GET /trip/wisata/:wisataId`
- `apiBooking`:
 - `create(payload)` → `POST /booking`
 - `getAll()` → `GET /booking`
 - `updateStatus(id, payload)` → `PATCH /booking/:id` (digunakan untuk pembayaran booking)

5) Struktur Routing (Expo Router)

``user/src/app/_layout.tsx``

- Root layout menggunakan `Stack` dari `expo-router`.

- Bertugas merender konfigurasi navigasi stack.

`user/src/app/index.tsx`

- Halaman root `.`.
- Mengalihkan pengguna ke `/wisata`.

6) Screens / Halaman Utama

`user/src/app/wisata/index.tsx`

- Menampilkan daftar wisata.
- Memanggil `apiWisata.getAll()` saat mount.
- UI:
 - menampilkan card wisata
 - menampilkan gambar wisata bila `item.gambar` ada
 - menavigasi ke `/trip` saat card ditekan

`user/src/app/trip/index.tsx`

- Menampilkan daftar trip.
- Memiliki filter `Picker` berdasarkan `wisata`.
- Memanggil:
 - `apiTrip.getAll()` jika filter "Semua"
 - `apiTrip.getByWisata(wisataId)` bila filter spesifik
- Saat card trip ditekan:
 - navigasi ke `booking` dengan membawa `tripId`:
 - `router.push(`/booking?tripId=\${item.id}`)

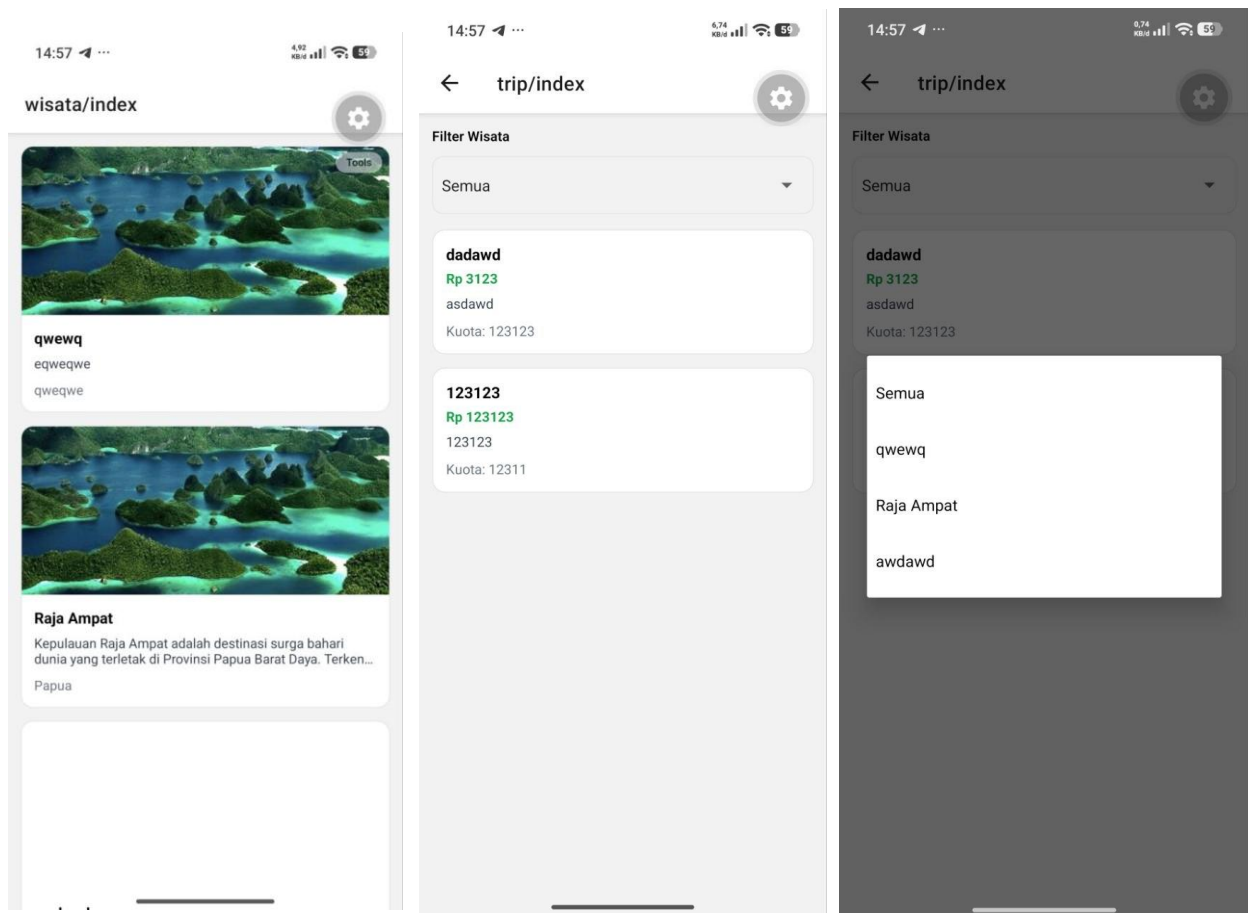
7) Tab Navigation

`user/src/app/(tabs)/\_layout.tsx`

- Mendefinisikan tab bar dengan `Tabs` dari `expo-router`.
- Tab yang tersedia:
 - `wisata`
 - `trip`
 - `booking`

- `my-bookings`
- Menentukan `tabBarIcon` dari `assets/images/tabIcons/\*`.

Dokumentasi Gambar User/Mobile



1. Gateway Module

Kode

```
import { Module } from '@nestjs/common';
import { GatewayController } from './gateway.controller';
import { WisataModule } from '../wisata/wisata.module';
import { TripModule } from '../trip/trip.module';
import { BookingModule } from '../booking/booking.module';
```

```
@Module({
  imports: [
    WisataModule,
    TripModule,
    BookingModule,
  ],
  controllers: [GatewayController],
  providers: [],
})
```

```
})  
export class GatewayModule {}
```

Fungsi Gateway Module

Gateway Module bertugas:

Mengimpor modul Wisata.

Mengimpor modul Trip.

Mengimpor modul Booking.

Mendaftarkan Gateway Controller.

Dengan konfigurasi tersebut, Gateway dapat menggunakan seluruh service yang terdapat pada ketiga modul tersebut.

2. Gateway Controller

Kode Controller

```
@Controller('api')  
export class GatewayController
```

Bagian ini menentukan bahwa seluruh endpoint diawali dengan:

/api

Contoh:

```
/api/wisata  
/api/trip  
/api/booking
```

3. Endpoint Wisata

Menambah Wisata

Endpoint

POST /api/wisata

Fungsi

Menambahkan data wisata baru ke database.

Request Body

```
{  
  "nama": "Kawah Putih",
```

```
"lokasi": "Bandung",  
"deskripsi": "Wisata alam"  
}
```

Menampilkan Semua Wisata

Endpoint

GET /api/wisata

Fungsi

Mengambil seluruh data wisata yang tersedia.

Menampilkan Detail Wisata

Endpoint

GET /api/wisata/:id

Contoh

GET /api/wisata/1

Fungsi

Mengambil detail wisata berdasarkan ID.

Mengubah Data Wisata

Endpoint

PATCH /api/wisata/:id

Fungsi

Memperbarui data wisata berdasarkan ID.

Menghapus Wisata

Endpoint

DELETE /api/wisata/:id

Fungsi

Menghapus data wisata dari database.

4.Endpoint Trip

Menambah Trip

Endpoint

POST /api/trip

Fungsi

Menambahkan paket trip baru.

Menampilkan Semua Trip

Endpoint

GET /api/trip

Fungsi

Mengambil seluruh data trip.

Menampilkan Trip Berdasarkan Wisata

Endpoint

GET /api/trip/wisata/:wisataId

Contoh

GET /api/trip/wisata/2

Fungsi

Mengambil daftar trip berdasarkan wisata yang dipilih.

Menampilkan Detail Trip

Endpoint

GET /api/trip/:id

Fungsi

Mengambil detail trip berdasarkan ID.

Mengubah Data Trip

Endpoint

PATCH /api/trip/:id

Fungsi

Memperbarui data trip.

Menghapus Trip

Endpoint

DELETE /api/trip/:id

Fungsi

Menghapus data trip dari sistem.

5.Endpoint Booking

Membuat Booking

Endpoint

POST /api/booking

Fungsi

Membuat pemesanan wisata baru.

Request Body

```
{  
  "namaUser": "Aldo",  
  "email": "aldo@gmail.com",  
  "jumlahTiket": 2,  
  "tripId": 1  
}
```

Menampilkan Semua Booking

Endpoint

GET /api/booking

Fungsi

Menampilkan seluruh data booking yang tersimpan.

Mengubah Status Booking

Endpoint

PATCH /api/booking/:id

Contoh

PATCH /api/booking/1

Request Body

```
{  
  "status": "paid"  
}
```

Fungsi

Mengubah status booking.

Contoh perubahan status:

pending → paid

Tampilan Akhir dari tampilan User

